

Repository Insight Analysis: RQ2 - Which parts of the codebase evolve the most, and how does file-level activity vary across the project structure?

BY: Hafsa SAIH

I. Research Question and Motivation

This study analyzes the NumPy repository to identify which components are modified most often and carry the highest code churn, and how this activity distributes across the project's structure over time.

NumPy is well suited to this analysis. It has over 23 years of version-controlled history beginning in December 2001. It went through several major architectural transitions including its separation from SciPy, the Python 3 migration, and the NumPy 2.0 reorganization in 2023-2024. It has more than 2,000 recorded contributor identities and is a dependency of almost every scientific Python library.

The unit of observation we used is one file change within one commit. Each row in the dataset represents one file being modified in one specific commit. This structure preserves file-level granularity while allowing aggregation by component, time period, or individual file.

II. Data Extraction

Data was extracted using PyDriller. The GitHub REST API wasn't used because it imposes a rate limit of 5,000 requests per hour. NumPy has over 30,000 commits, and obtaining file-level detail for each would require at least one API call per commit. Full extraction within the time constraint we had would not be very feasible by that route. Local extraction also requires no authentication, which simplifies replication.

Several cleaning decisions were made before analysis. Merge commits were excluded because their diffs double-count changes already captured in the constituent commits. File records where both added and deleted lines equal zero were excluded because git cannot compute line counts for binary files, and retaining them adds noise without useful information.

The most substantive cleaning decision concerned auto-generated and vendored files. Inspection of the 20 highest-churn file changes revealed that all of them were either machine-generated C translations of Fortran source code, documentation book artifacts, or files from NumPy's early life under the SciPy namespace. The largest single file change was a 97,895-line addition to `dlapack_lite.c`, a Fortran-to-C translation generated by the `f2c` tool rather than written by a human developer. Including these files would cause auto-generated artifacts to dominate the hotspot ranking and would not answer the research question. Files matching the identified patterns were removed before analysis.

The primary analysis focuses on source code files with extensions `.py`, `.c`, `.h`, `.pyx`, `.pxd`, and `.cpp`. Documentation and configuration files were retained in the full saved dataset but excluded from the hotspot computations.

Two derived metrics were computed. Total churn is defined as added lines plus deleted lines for a given file or component within a given period. Churn density is total churn divided by lifespan in years, clipped at a minimum of 0.5 years. Raw modification counts favor older

files regardless of how intensively they were modified. Churn density normalizes for file age and allows comparison across files with different lifespans.

In the heatmap visualization, each component's annual churn was divided by that component's own maximum value, producing a relative scale from 0 to 1 per row. Without this normalization, `numpy/core` would dominate every column and the heatmap would convey no more information than the bar chart. The tradeoff is that absolute cross-component comparison is not possible from the heatmap, but that comparison is covered by the bar chart.

III. Results and Interpretation

The most consistent finding across all four visualizations is the dominance of `numpy/core`. This component accumulated 885,002 total churn, which is 4.95 times the churn of the second-ranked component `numpy/f2py` at 178,668. The gap between rank 1 and rank 2 is larger than the combined churn of ranks 3 through 15.

This concentration reflects the difficulty of maintaining the core array abstraction. The array object, memory layout, broadcasting rules, dtype system, and C-level operation dispatch all reside in this component. Each subsystem requires sustained modification as the Python ecosystem evolves and API commitments change across major versions.

A related finding concerns `numpy/core` and `numpy/_core`. In NumPy 2.0, the core module was reorganized and its contents moved to `numpy/_core`. These two entries appear separately in the dataset because they have distinct file paths. Together they account for 1,029,051 lines of combined churn. The `numpy/_core` component accumulated 144,049 churn in approximately two years, while `numpy/core` accumulated 885,002 over more than 20 years. This implies a substantially higher per-year modification rate during the recent reorganization, consistent with the scale of breaking changes introduced in NumPy 2.0.

Beyond core dominance, the temporal visualizations show that each component tells a distinct story. `numpy/f2py` shows intense activity in 2007-2009 followed by near-zero activity for 15 years. Once the Fortran interfacing module reached a stable state, it stopped requiring major changes. `numpy/random` shows a single sharp peak in 2019, corresponding to the complete redesign of the random number generation system in NumPy 1.17. The bounded, single-year nature of the peak reflects a planned coordinated rewrite rather than incremental maintenance. `numpy/distutils` shows a clean flatline after 2022 when the custom build system was deprecated in favor of Meson. `numpy/typing` does not appear in the data before 2020, then shows immediate sustained activity, reflecting how quickly the maintainers moved once type annotation tooling matured.

At the file level, the scatter plot reveals a distinction that component-level aggregation cannot show. High-modification files fall into two categories. Chronic hotspots are files modified continuously over long lifespans. `test_multiarray.py` has 821 modifications over 17.7 years. `function_base.py` has 708 modifications over 17.6 years. `ma/core.py` has 682 modifications over 18.6 years. Their sustained modification reflects accumulated responsibility. Short-lived intensive files have shorter lifespans but very high churn density. `arrayobject.c` existed for 3.3 years with a churn density of 27,897 lines per year. These files were created for a specific architectural purpose, modified intensively, and then replaced. They represent scaffolding during an active restructuring.

Chronic hotspots and short-lived intensive files represent different risk environments. A developer modifying a chronic hotspot works on a file with decades of accumulated design decisions. A developer modifying a short-lived intensive file is likely operating in the middle of an active architectural transition. AI coding agents currently lack awareness of either type of risk.

A final observation concerns test files. Six of the top 20 most frequently modified files are test files, and `test_multiarray.py` is the single most modified file in the repository. This reflects a development culture where tests are treated as living documentation of intended behavior. Each time an API contract changes, the corresponding tests change with it. An agent that modifies implementation files without corresponding test updates is likely to produce pull requests that fail review, which is consistent with findings in the literature on agentic pull request rejection.

IV. Validity Assessment

Several threats to validity affect this analysis. The most significant is the file rename problem. When a file is renamed in git, it appears as two distinct entries in the modification history. Churn attributed to each path underestimates the true accumulated modification of the underlying component. NumPy's architectural transitions involved substantial file reorganization, so this effect is present throughout the dataset. The `numpy/core` and `numpy/_core` split is the most prominent example. This analysis does not track renames across paths, which is a known limitation.

Author identity conflation is a second concern. The dataset records 2,259 unique author identities, but many represent the same person using different email addresses across machines. The actual number of unique human contributors is lower. This does not affect the file-level hotspot analysis but would affect any contributor-level analysis.

The focus on source code files means that components maintained primarily through documentation or configuration changes may appear less active than they are. The full dataset is saved and available for extended analysis. At the temporal boundaries, the earliest years are recorded under `scipy/` path prefixes from before NumPy's independent existence and are treated as historical artifacts. The most recent month in the dataset is incomplete at extraction time, creating a small boundary effect in the rolling average.

Finally, churn measures the volume of change, not its nature. A large churn value could reflect a major improvement, a bulk reformatting, or a large deletion. This analysis treats all churn as equivalent, which is the standard simplification in repository mining research.

V. Follow-up Research Question

This analysis identifies hotspots by modification volume. It does not distinguish between modifications driven by defect correction and those driven by feature addition or refactoring. A natural extension would ask whether structural hotspots in NumPy show disproportionately high defect rates, and whether this relationship differs between chronic hotspots and short-lived intensive files.

This question could be addressed by joining the file-level data from this study with issue and pull request data from the GitHub API. Bug-fixing commits could be identified using conventional commit message patterns such as the fix: prefix, or by linking commits to GitHub issues carrying a bug label. The two-category classification introduced here would serve as the primary stratification variable. The hypothesis is that chronic hotspots, which accumulate responsibility over long periods, show higher defect association rates than short-lived intensive files, which represent planned architectural work.

VI. AI Use Transparency

ChatGPT was used at some points during this assignment. Before writing any code, I asked conceptual questions about git internals, the difference between the GitHub API and local extraction with PyDriller, and why merge commits are excluded in repository mining research. These questions helped me make informed methodological decisions before starting the extraction. During setup, I used it to diagnose a kernel registration conflict in JupyterLab on Windows. During the coding process, I asked targeted implementation questions about data structure choices and binary file handling. For the time series visualization, I asked about standard smoothing approaches for spiky monthly data. Before writing the validity section, I asked it to outline standard threats to validity in repository mining research, which I used as a checklist before writing that section myself.

For Python syntax and standard library functions, I used Google search and the official documentation throughout. This included pandas, matplotlib, and os.path references.

The full session is attached as a PDF. All analytical decisions, including the choice of research question, the exclusion of vendored files, the churn density metric, and the interpretation of every visualization, were mine.